7-Day Capstone: Meridian Commerce

Building an End-to-End Intelligent Retail Assistant

GenAI Mastery Bootcamp | E-Commerce Industry Track | Progressive Project, Days 1-7

The Use Case

Meridian Commerce is a mid-sized online retailer selling electronics, footwear, apparel, and home goods. Like most growing e-commerce companies, they're facing three real, industry-standard problems at once:

- Rising return rates are cutting into margins, and nobody can predict which orders are actually at risk before they ship.
- The support team is overwhelmed with repetitive questions about orders, returns, and policies — most of it doesn't need a human.
- The product catalog has grown too large for keyword search alone to work well, and customers can't find what they're actually looking for.

You've been hired as the AI engineer to build Meridian's first end-to-end intelligent retail assistant — a system that predicts return risk, understands customer language, answers questions grounded in real company policy, and searches the catalog by meaning, not just keywords.

This is intentionally the same shape of problem real retail, banking, and healthcare companies solve with GenAI today — not a toy exercise. Each day of the bootcamp builds one genuine piece of this system, using the same datasets throughout.

How Each Day Builds Toward This

This is a map, not a solution — how you actually build each piece is yours to figure out, using what that day teaches.

Day 1 — ML Foundations

Predict which orders are likely to be returned, using the Orders & Returns dataset. A genuine classification problem with class imbalance — the same kind of task that opened Day 1.

Day 2 — LLM Fundamentals

Use embeddings to power semantic product search over the Product Catalog, and benchmark a few models on cost, latency, and quality for this specific task.

Day 3 — Prompt Engineering

Extract structured signals from the Customer Reviews and Support Tickets — sentiment, complaint category — and classify incoming tickets by urgency using zero-shot and few-shot prompting. Also use data analysis prompting on the Orders & Returns dataset: EDA over return patterns, KPI extraction (e.g. return rate by category), and SQL generation against the orders table.

Day 4 — Advanced Prompting

Build reasoning for ambiguous cases — e.g., should a high-value flagged return be auto-approved — using Chain of Thought and Tree of Thought, with a System Prompt enforcing compliance boundaries. Use Prompt Chaining to break multi-step ticket triage into stages (summarize → categorize → recommend action). Apply Self-Reflection to have the assistant critique its own draft responses to customers before sending. Apply Ethical and Negative Prompting when generating customer-facing messages about denied returns or policy enforcement, so the tone stays fair and never manipulative or alarming.

Day 5 — LangChain, Memory & Streaming

Turn the assistant into an actual conversational interface using LangChain — Prompt Templates and LCEL Chains connecting the pieces from earlier days. Add memory across turns so a customer doesn't repeat their order number,

streaming responses so replies feel responsive, and caching for repeated policy questions. Build the actual interface itself in Streamlit, using the Support Tickets dataset to simulate real conversations.

Day 6 — RAG Foundations

Ground the assistant's policy answers in Meridian's real Return, Shipping, and Warranty policy documents — not invented answers.

Day 7 — Vector DB & Retrieval

Scale product search with FAISS/Chroma, add hybrid search so exact SKU/order lookups work alongside natural-language search, and filter by category or price.

The Datasets

Five related datasets, sharing common IDs (order_id, product_id, customer_id) so they connect into one coherent system — exactly like a real company's data would.

1. Orders & Returns (structured/tabular)

Used primarily in Day 1, and again as ground truth throughout.

Field	Description
order_id	Unique order identifier
customer_id	Unique customer identifier
product_id	Links to the Product Catalog
category	Product category at time of order
price	Order price (post-discount)
discount_pct	Discount applied, if any
order_date	Date the order was placed
delivery_days	Days from order to delivery
payment_method	e.g. card, UPI, wallet, COD
is_returned	Target variable: 1 if returned, 0 if not
return_reason	Free text, populated only if returned

2. Product Catalog (text + metadata)

Used in Day 2, Day 6, and Day 7.

Field	Description
product_id	Unique product identifier
sku	Exact product code — deliberately included so Day 7's hybrid search has a real reason to exist
product_name	Short product title
description	Longer free-text description — the field you'll embed
category / sub_category	Used for metadata filtering
brand	Used for metadata filtering
price	Used for metadata filtering

stock_status	In stock / out of stock
avg_rating	Average customer star rating

3. Customer Reviews (unstructured text)

Used in Day 2 and Day 3.

Field	Description
review_id	Unique review identifier
product_id	Links to the Product Catalog
customer_id	Links to Orders
review_text	The actual review, free text
star_rating	1-5
review_date	Date submitted

4. Support Tickets / Conversations (unstructured, conversational)

Used in Day 3, Day 4, and Day 5.

Field	Description
ticket_id	Unique ticket identifier
customer_id	Links to Orders
order_id	Optional — not every ticket is order-specific
message_text	The customer's message, free text
channel	Chat, email, or phone-transcribed
timestamp	When the message was sent
resolution_status	Open, resolved, escalated

5. Policy & FAQ Documents (long-form text)

Used in Day 6 and Day 7 — the actual RAG knowledge base.

- Return & Refund Policy
- Shipping & Delivery Policy
- Warranty Policy
- Payment & Refunds FAQ
- Loyalty Program Terms

Where to Get the Data

Recommended: the Olist Brazilian E-Commerce dataset (public, real data)

A genuinely real public dataset with orders, products, customers, reviews, and payments already connected by shared IDs — an excellent match for this project's structure, and one you've used successfully in prior enterprise training. Available on Kaggle. You'll need to rename/remap a few columns to match the field names

above, and you'll need to write the Policy & FAQ documents and Support Ticket messages yourself, since Olist doesn't include those.

Alternative: generate a synthetic version of all five datasets — including the Support Tickets and Policy documents — directly using Claude, matching the exact schemas above. This is a legitimate, fast option if sourcing and cleaning real data would eat into time better spent on the actual bootcamp skills.

A note on scope

This document deliberately stops at the use case and the datasets. Everything else — how you engineer features, which chunking strategy fits the policy documents, how you structure the RAG pipeline, what the conversational assistant actually looks like — is yours to design, day by day, using what each session teaches.